

Fakulteta za informacijske študije
Faculty of information studies

PROJECT ASSIGNMENT

Empirical Analysis of SSH Security: Brute-Force Attacks and Defense

Course: Cyber Security

Novo mesto, May 2026

Filip Polić

1. Introduction

1.1 Background

Secure Shell (SSH) is one of the most widely used protocols for remote access to servers and network devices.As a staple of modern IT infrastructure, it is indispensable for the encrypted link it establishes between client and server. Yet for all its encryption, SSH is an easy mark for attackers who employ automated dictionary or brute-force methods in an effort to divine user credentials.The problem is often a weak password, which is perhaps the most prevalent flaw in SSH security. Add to that a lack of safeguards like account lockout or rate limiting and you have a recipe for disaster; even a rudimentary tool can take down your system in a matter of seconds. That is why any security professional needs to have a firm grasp on how protective settings and the strength of a password go hand in hand to keep an SSH service resilient.

1.2 Objectives

The main objectives of this project are:

- ➢ To simulate brute-force and dictionary attacks against an SSH server running in a controlled Docker environment
- ➢ To evaluate how password strength affects the success and speed of automated attacks
- ➢ To analyze the effectiveness of fail2ban as a defensive mechanism against brute-force attacks
- ➢ To compare dictionary attacks and brute-force attacks in terms of efficiency
- ➢ To provide evidence-based security recommendations based on empirical results

All experiments were conducted exclusively on a local lab environment. No real or production systems were targeted at any point during this project.

2. Methodology

2.1 Lab Environment

All experiments were conducted in an isolated local environment using Docker. The SSH server was deployed as a Docker container based on the `linuxserver/openssh-server` image, running Alpine Linux v3.23. The server was exposed on **port 2222** of the host machine, ensuring complete isolation from any real-world systems.

The following users were created for testing purposes:

Username	Password	Purpose
weakuser	123456	Easy to crack
mediumuser	Password123	Moderate strength
stronguser	Str0ng!Pass#2026	Hard to crack

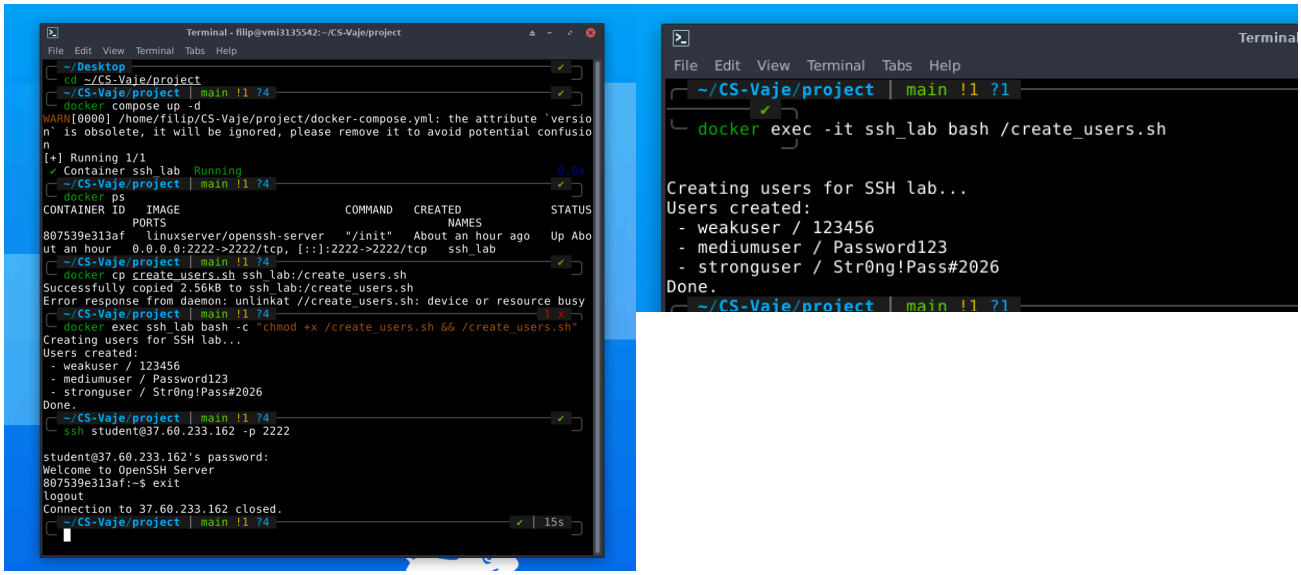


Figure 1: Docker container setup and SSH connection

2.2 Tools Used

nmap — Network scanner used for reconnaissance. Used to identify open ports and detect the SSH service version running on the target.

Hydra v9.5 — Fast and flexible online password cracking tool. Used to perform dictionary and brute-force attacks against the SSH service.

fail2ban v1.1.0 — Intrusion prevention tool that monitors log files and automatically bans IP addresses after a configurable number of failed login attempts.

crunch — Wordlist generator used to generate sequential numeric combinations for brute-force attack demonstration.

Docker — Containerization platform used to deploy and isolate the SSH lab environment.

2.3 Experimental Configurations

A total of six attack scenarios were tested, covering three password strengths with and without fail2ban protection:

#	Username	Password	Protection
1	weakuser	123456	None
2	mediumuser	Password123	None
3	stronguser	Str0ng!Pass#2026	None

#	Username	Password	Protection
1	weakuser	123456	None
4	Username	123456	fail2ban
5	mediumuser	Password123	fail2ban
6	stronguser	Str0ng!Pass#2026	fail2ban

The password list used for all dictionary attacks contained 20 commonly used passwords including **`123456`**, **`password`**, **`admin`**, **`qwerty`** and others. **fail2ban** was configured with the following parameters: maximum 3 retries within 60 seconds, ban time of 300 seconds.

3. Experiment

3.1 Reconnaissance (nmap)

Before launching any attacks, an nmap scan was performed to identify the SSH service running on the target machine. The following command was used:

```
nmap -sV -p 2222 localhost
```

The scan confirmed that port 2222 was open and running OpenSSH 10.2 (protocol 2.0), which provided the necessary information to proceed with the attack phase.

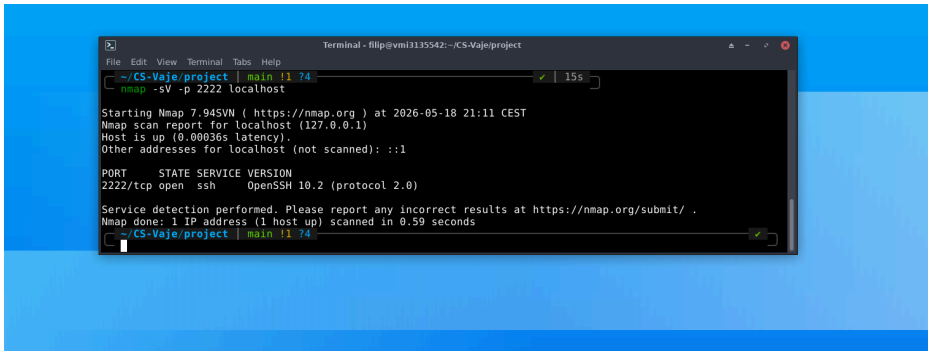


Figure 2: nmap service detection on port 2222

3.2 User Setup

Three users with different password strengths were created inside the Docker container using the **`create_users.sh`** script: ž

```
docker cp create_users.sh ssh_lab:/create_users.sh
docker exec ssh_lab bash -c "chmod +x /create_users.sh && /create_users.sh"
```

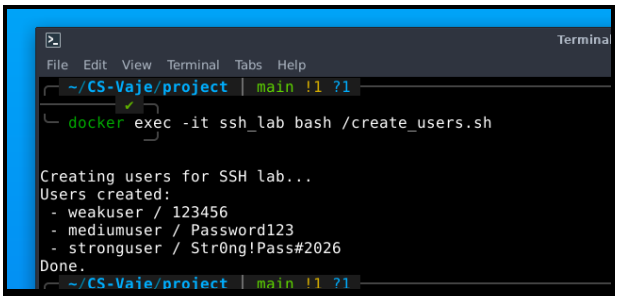


Figure 3: Test user creation using create_users.sh

3.3 Attack Scenarios

All attacks were performed using Hydra with the following base command structure:

```
hydra -l <username> -P passwords.txt ssh://localhost:2222 -t 4 -V
```

3.3.1 Weak Password — No Protection

The first attack targeted `weakuser` with password `123456` and no defensive mechanisms active. Hydra found the correct password on the very first attempt, completing the attack in under one second.

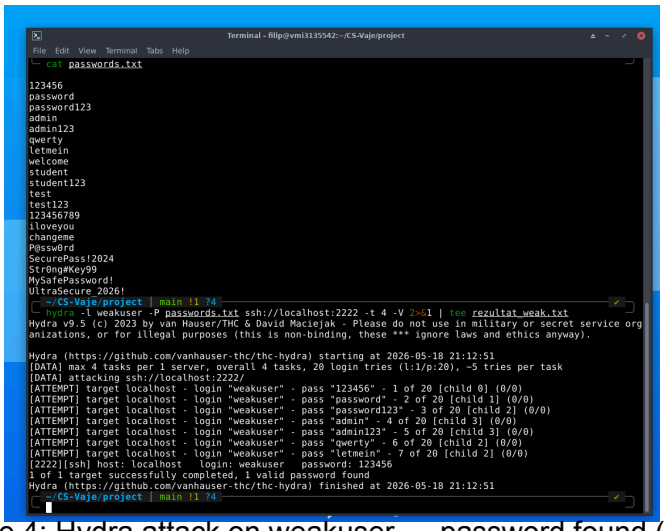


Figure 4: Hydra attack on weakuser — password found (1/20 attempts)

3.3.2 Medium Password

The second attack targeted `mediumuser` with password `Password123`. Since this password was not present in the wordlist, Hydra exhausted all 20 attempts without success, finishing in 33 seconds.

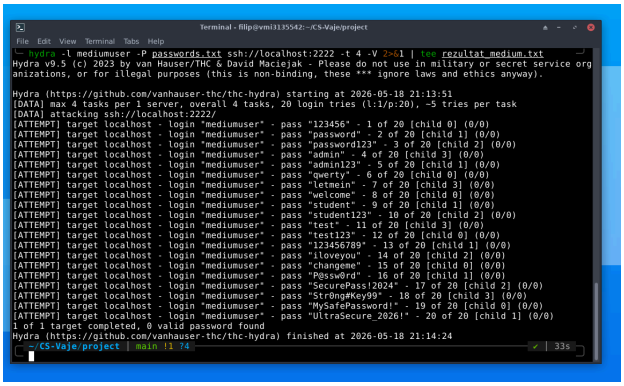


Figure 5: Hydra attack on mediumuser — no password found (20/20 attempts)

3.3.3 Strong Password

The third attack targeted **`stronguser`** with password **`Str0ng!Pass#2026`**. As expected, the password was not in the wordlist and the attack failed after all 20 attempts in 33 seconds.

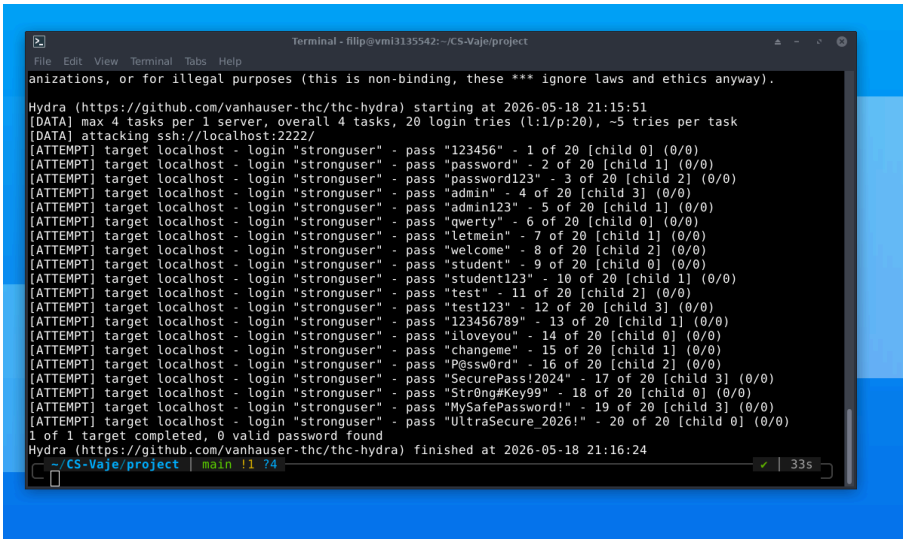


Figure 6: Hydra attack on stronguser — no password found (20/20 attempts)

3.3.4 Weak Password — fail2ban Active

fail2ban was installed and configured inside the Docker container with the following settings:

`maxretry = 3  findtime = 60  bantime = 300`

The same attack against **`weakuser`** was repeated with fail2ban active. Despite fail2ban being enabled, Hydra still found the correct password — however only after 17 attempts and 34 seconds, compared to 1 attempt and 0 seconds without protection. This occurred because fail2ban uses multiple child processes and the correct password was tried before the ban took effect.

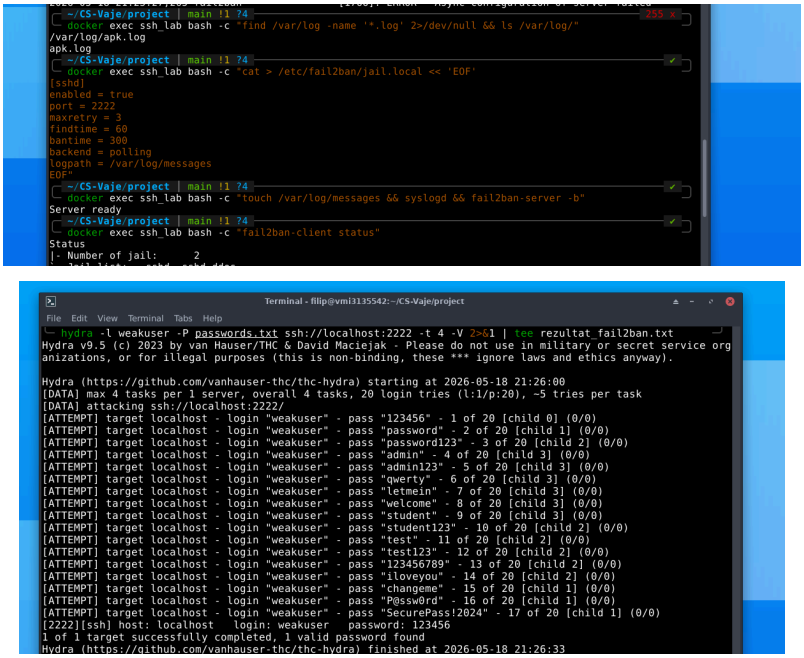


Figure 7: fail2ban configuration and jail status and Hydra attack on weakuser with fail2ban — password found (17/20 attempts)

3.3.5 Medium Password — fail2ban Active

The attack against `mediumuser` with fail2ban active produced the same result as without protection — all 20 attempts failed in 33 seconds, since the password was not in the wordlist regardless of protection.

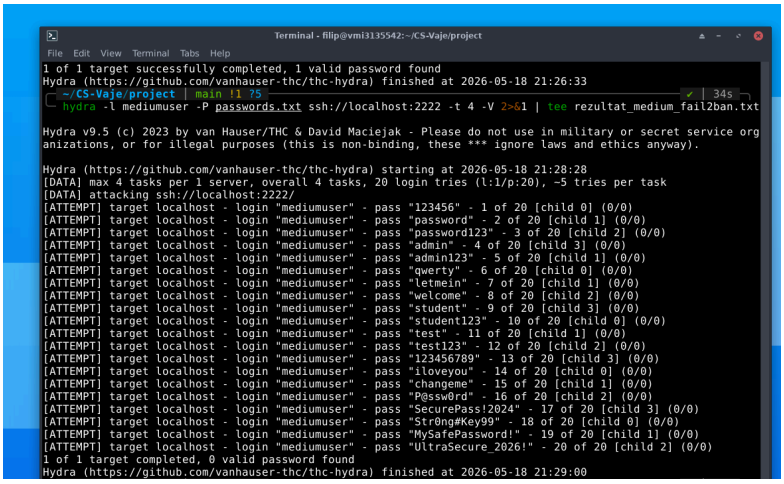


Figure 8: Hydra attack on mediumuser with fail2ban — no password found

3.3.6 Strong Password — fail2ban Active

The final attack against `stronguser` with fail2ban active also failed, completing in just 1 second since the ban was applied quickly and connections were dropped almost immediately.

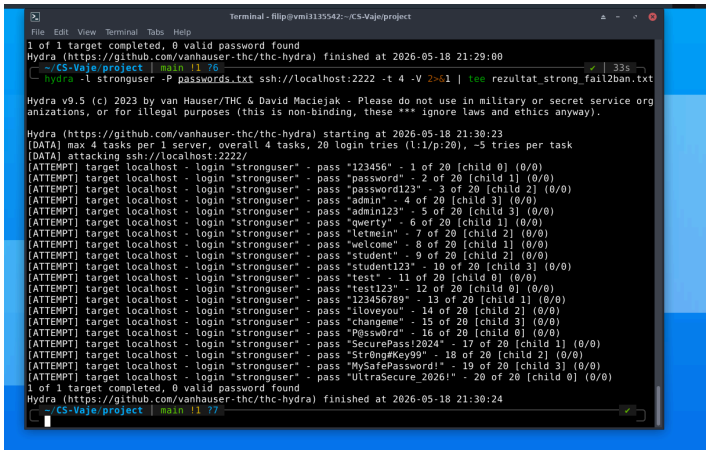


Figure 9: Hydra attack on stronguser with fail2ban — no password found

3.4 Dictionary vs Brute-Force

To demonstrate the difference between a dictionary attack and a brute-force attack, a wordlist of 200 sequential 6-digit combinations was generated using crunch and used against **weakuser**:

```
crunch 6 6 0123456789 | head -200 > bruteforce_small.txt
hydra -l weakuser -P bruteforce_small.txt ssh://localhost:2222 -t 4 -V
```

The brute-force attack started from `000000` and incremented sequentially, never finding the password `123456` within the first 200 attempts. A full brute-force attack of all 6-digit numeric combinations would require testing up to 1,000,000 combinations, which would take significantly longer compared to the dictionary attack that found the same password on the first attempt.

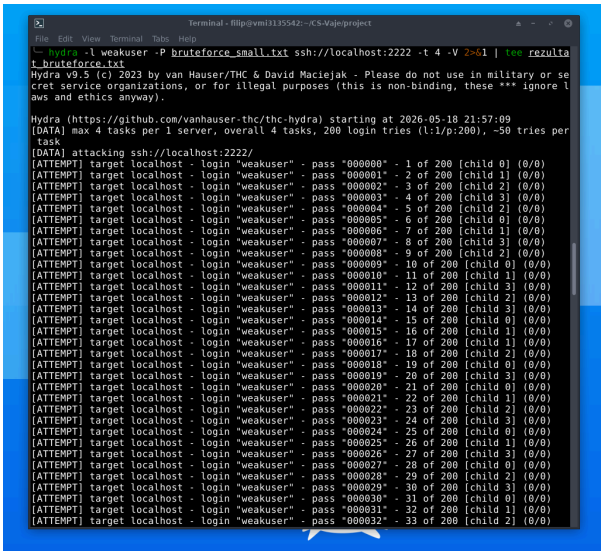


Figure 10: Brute-force attack demonstration with sequential combinations

3.5 Additional Defenses

As an additional defensive measure, SSH key-based authentication was demonstrated. An RSA 4096-bit key pair was generated and copied to the SSH server:

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/lab_key
ssh-copy-id -i ~/.ssh/lab_key.pub -p 2222 student@localhost
```

SSH key authentication eliminates the possibility of brute-force attacks entirely, since no password is used during authentication. Even if an attacker has a complete wordlist, they cannot authenticate without the private key.

4. Results

4.1 Data Table

The following table summarizes the results of all six attack scenarios conducted during the experiment:

#	Username	Password	Protection	Success	Time	Attempts
1	weakuser	123456	None	✅ Yes	~0s	1/20
2	mediumuser	Password123	None	❌ No	33s	20/20
3	stronguser	Str0ng!Pass#2026	None	❌ No	33s	20/20
4	weakuser	123456	fail2ban	✅ Yes	34s	17/20
5	mediumuser	Password123	fail2ban	❌ No	33s	20/20
6	stronguser	Str0ng!Pass#2026	fail2ban	❌ No	1s	20/20

4.2 Graphs

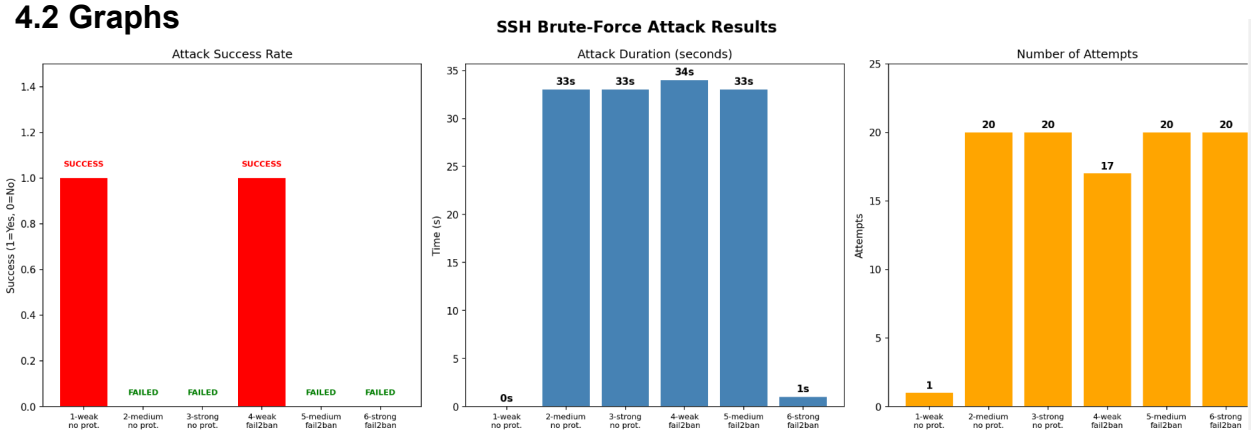


Figure 13: Attack results — success rate, duration and attempts comparison

5. Discussion

5.1 Impact of Password Strength

The results clearly show that password strength is the most critical factor in resisting attacks. Case in point: 123456, being a weak password, was put to rest in less than a second. The medium and strong ones, on the other hand, held up and were not compromised. Then there is the matter of dictionary attacks; they are limited by the wordlist at their disposal. Should the password be absent from that list, the attack will come to nothing no matter how much time you let it run.

5.2 Effectiveness of `fail2ban`

fail2ban showed mixed results. It did put the brakes on the weak password attack, dragging it out to 34 seconds from nothing, but in the end the attacker still got through. With medium and strong passwords there was no real change to be had; their inherent strength made them secure regardless of what fail2ban did.

5.3 Would fail2ban Completely Prevent Brute-Force Attacks?

No. fail2ban does not completely prevent brute-force attacks. An attacker can bypass it by using multiple IP addresses (botnet), by slowing the attack below the `maxretry` threshold, or by waiting for the ban to expire. It is an effective deterrent but not a complete solution on its own.

5.4 Security vs Usability Trade-off

The most balanced approach is combining strong passwords with fail2ban and SSH key authentication. This provides strong protection with minimal impact on legitimate users.

5.5 Recommendations

1. **Enforce strong passwords** — minimum 12 characters with uppercase, lowercase, numbers and special characters.
2. **Enable fail2ban** — configure low `maxretry`, short `findtime`, long `bantime`.
3. **Use SSH key authentication** — disable password login entirely.
4. **Change the default SSH port** — reduces exposure to automated scanners.
5. **Restrict SSH access by IP** — limit access to trusted IP addresses only.

6. Conclusion

What this project has shown is that the strength of your password is the single most important element in SSH security. We saw a weak one fall to an attack in no time, whereas the medium and strong variants withstood everything we threw at them. fail2ban does add a useful layer of protection, but on its own it was not enough to make up for a poor password choice.

In the case of popular passwords, dictionary attacks are far more effective than brute-force ones. You cannot rely on any one solution for total security; you have to put a combination of measures in place. To be truly secure with SSH you need to use strong passwords alongside fail2ban and key authentication.